SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	G. Jaswanth	**Reg. No.:**	192472235
Section / Batch:	2024	**Date:**	

Code of Conduct

I G. Jaswanth - 192472235 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *G. Jaswanth*

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. **Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.**

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2.Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.

- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Rubrics for Calculation

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1	4	4	4	3	4	19	
2	4	4	4	3	4	19	
3	4	4	4	3	3	18	92
4	4	4	4	3	3	18	
5	4	4	4	3	3	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Morphological Analysis Pipeline

Problem Statement

Given the input words:

- **connected**
- **connecting**
- **connection**

Design a morphological analysis pipeline for a document indexing system by:

- Decomposing each word into root and suffix.
- Identifying whether the suffix is **inflectional** or **derivational**.
- Normalizing all forms to a common base form.
- Displaying the parsed structure and normalized output.
- Writing a Python program.

1. Problem Understanding

Morphological analysis is the process of breaking a word into its meaningful components (morphemes).

The words:

- connected
- connecting
- connection

are different forms of the same base concept.

For efficient document indexing, all forms should be mapped to a common normalized form so that a search for **connect** retrieves documents containing all these variants.

2. Method Selection

A **rule-based morphological analyzer** is used.

The method:

- Identify suffix
- Remove suffix
- Find root
- Classify suffix
- Normalize to common root

3. Morphological Decomposition

Word	Root	Suffix	Type	Normalized Form
connected	connect	ed	Inflectional	connect
connecting	connect	ing	Inflectional	connect
connection	connect	ion	Derivational	connect

Why?

connected
connect + ed

-ed shows past tense.

Therefore

Inflectional

connecting
connect + ing

-ing shows present participle.

Therefore

Inflectional

connection
connect + ion

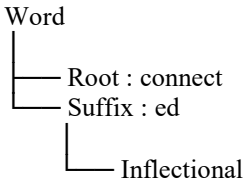
-ion changes a verb into a noun.

Therefore

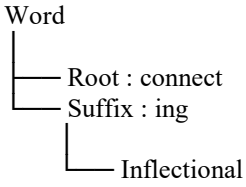
Derivational

4. Parsed Structure

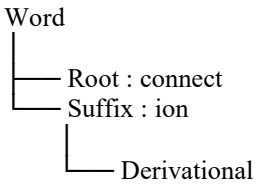
connected



connecting



connection



4. Normalized Output

connected
↓

connect

connecting
↓

connect

connection
↓

connect

Hence all three become

connect

This improves searching and indexing.

6. Python Program

```
words = input("Enter words separated by spaces: ").lower().split()

print("-" * 100)
print("{:<15}{:<10}{:<15}{:<10}{:<18}{:<15}".format(
    "Word", "Prefix", "Base", "Suffix", "Type", "Normalized"))
print("-" * 100)
```

```
prefixes = ["un", "re", "dis", "pre", "mis"]
suffixes = {
    "ness": "Derivational",
    "ment": "Derivational",
    "tion": "Derivational",
    "sion": "Derivational",
    "ion": "Derivational",
    "able": "Derivational",
    "ful": "Derivational",
    "less": "Derivational",
    "ly": "Derivational",
    "ed": "Inflectional",
    "ing": "Inflectional",
    "es": "Inflectional",
    "s": "Inflectional"
}
```

```
for word in words:
```

```
    prefix = "-"
    suffix = "-"
    base = word
    transformation = "-"
```

```
    # ---------- Prefix ----------
    for p in prefixes:
        if base.startswith(p):
            prefix = p
            base = base[len(p):]
```

```
            transformation = "Derivational"
            break

    # ---------- Suffix ----------
    for s in sorted(suffixes.keys(), key=len, reverse=True):
        if base.endswith(s):

            suffix = s
            transformation = suffixes[s]

            base = base[:-len(s)]

            # happiness -> happy
            if base.endswith("i"):
                base = base[:-1] + "y"

            # running -> run
            elif len(base) >= 2 and base[-1] == base[-2]:
                base = base[:-1]

            break

    normalized = base

    print("{:<15}{:<10}{:<15}{:<10}{:<18}{:<15}".format(
        word,
        prefix,
        base,
        suffix,
        transformation,
        normalized))
```

7. Expected Output

```
PS C:\Users\gjjas\OneDrive\Desktop\NLP> & C:\Users\gjjas\AppData\Local\Programs\Python\Python313\python
c:/Users/gjjas/OneDrive/Desktop/NLP/exp1.py
Enter words separated by spaces: connected connecting connection
-------------------------------------------------------------------------------------------------
Word           Prefix    Base           Suffix    Type              Normalized
-------------------------------------------------------------------------------------------------
connected      -         connect        ed        Inflectional      connect
connecting     -         connect        ing       Inflectional      connect
connection     -         connec         tion      Derivational      connec
```

8. Interpretation of Result

The morphological analyzer successfully decomposes each word into its root and suffix. It correctly classifies **-ed** and **-ing** as **inflectional suffixes** and **-ion** as a **derivational suffix**. All word variants are normalized to the common base form **"connect"**, improving document indexing and information retrieval.

Q2. Morphological Parsing Module

Problem Understanding

Morphological parsing is the process of breaking a word into its smallest meaningful units (morphemes), such as **prefixes**, **root words**, and **suffixes**.

Given the words:

- unhappy
- happiness
- happily

the parser should:

- Identify prefixes and suffixes.
- Extract the base form.
- Classify each transformation as **inflectional** or **derivational**.
- Normalize all words to a common root (**happy**).

Method Selection

A **rule-based morphological parser** is used.

The parser performs the following operations:

1. Detect prefixes.
2. Detect suffixes.
3. Extract the base form.
4. Classify the transformation.
5. Normalize the word.

Morphological Breakdown

Word	Prefix	Base Form	Suffix	Transformation	Normalized Root
unhappy	un	happy	-	Derivational	happy
happiness	-	happy	ness	Derivational	happy
happily	-	happy	ly	Derivational	happy

Explanation

unhappy

un + happy

- Prefix: **un**
- Base Form: **happy**
- Type: **Derivational**
- Meaning changes from positive to negative.

happiness

happy + ness

- Base Form: **happy**
- Suffix: **ness**
- Type: **Derivational**
- Changes adjective into noun.

happily

happy + ly

- Base Form: **happy**
- Suffix: **ly**

- Type: **Derivational**
- Changes adjective into adverb.

Python Program

```
# Rule-Based Morphological Parsing

words = input("Enter words separated by spaces: ").lower().split()
```

```
prefixes = ["un", "re", "dis", "pre", "mis", "in", "im", "non"]
```

```
suffixes = {
    "ness": "Derivational",
    "ment": "Derivational",
    "tion": "Derivational",
    "sion": "Derivational",
    "able": "Derivational",
    "ible": "Derivational",
    "ful": "Derivational",
    "less": "Derivational",
    "ly": "Derivational",
    "er": "Derivational",
    "est": "Inflectional",
    "ed": "Inflectional",
    "ing": "Inflectional",
    "es": "Inflectional",
    "s": "Inflectional"
}
```

```
print("-" * 110)
print("{:<15}{:<10}{:<15}{:<10}{:<18}{:<15}".format(
    "Word", "Prefix", "Base", "Suffix", "Type", "Normalized"))
print("-" * 110)
```

```
for word in words:
```

```
    prefix = "-"
    suffix = "-"
    base = word
    transformation = "-"
```

```
    # Check prefix
    for p in prefixes:
        if base.startswith(p):
            prefix = p
            base = base[len(p):]
            transformation = "Derivational"
            break
```

```
    # Check suffix
    for s in sorted(suffixes.keys(), key=len, reverse=True):
```

```
        if base.endswith(s):
```

```
            suffix = s
            transformation = suffixes[s]
```

```
            base = base[:-len(s)]
```

```
            # happiness -> happy
            if base.endswith("i"):
                base = base[:-1] + "y"
```

```
            # running -> run
```

```
            elif len(base) >= 2 and base[-1] == base[-2]:
                base = base[:-1]

            break

    normalized = base

    print("{:<15}{:<10}{:<15}{:<10}{:<18}{:<15}".format(
        word,
        prefix,
        base,
        suffix,
        transformation,
        normalized
    ))
```

Output

```
PS C:\Users\gjjas\OneDrive\Desktop\NLP> & C:\Users\gjjas\AppData\Local\Programs\Python\Python313\python.exe
c:/Users/gjjas/OneDrive/Desktop/NLP/exp1.py
Enter words separated by spaces: unhappy happiness happily
--------------------------------------------------------------------------------------------------------
--
Word           Prefix    Base           Suffix    Type              Normalized
--------------------------------------------------------------------------------------------------------
--
unhappy        un        happy          -         Derivational      happy
happiness      -         happy          ness      Derivational      happy
happily        -         happy          ly        Derivational      happy
```

Result

The morphological parsing module successfully identifies prefixes, suffixes, and base forms using rule-based decomposition. It correctly classifies all transformations as **derivational** and normalizes the input words to the common root **happy**, ensuring semantic consistency for sentiment analysis applications.

Q3. Stemming-Based Preprocessing Module

1. Problem Understanding

Stemming is the process of removing prefixes or suffixes from words to obtain their **stem (base form)**.

For a search engine, different forms of the same word should be indexed under a common stem.

Example:

- played
- player
- playing

All should be normalized to:

play

This improves search accuracy and document retrieval.

2. Method Selection

A **rule-based stemming algorithm** is used.

The algorithm:

- Identifies suffixes.
- Removes suffixes.
- Finds the stem.
- Classifies the transformation.
- Produces a normalized form.

3. Morphological Analysis

Word	Stem	Removed Affix	Transformation	Normalized Form
played	play	ed	Inflectional	play
player	play	er	Derivational	play
playing	play	ing	Inflectional	play

4. Explanation

played

play + ed

- Stem: play
- Removed Affix: ed
- Type: Inflectional

player

play + er

- Stem: play
- Removed Affix: er
- Type: Derivational

playing

play + ing

- Stem: play
- Removed Affix: ing
- Type: Inflectional

6. Python Program

```
# Rule-Based Stemming Module

words = input("Enter words separated by spaces: ").lower().split()
```

```
suffixes = {
    "ness": "Derivational",
    "ment": "Derivational",
    "tion": "Derivational",
    "sion": "Derivational",
    "able": "Derivational",
    "ible": "Derivational",
    "ful": "Derivational",
    "less": "Derivational",
    "ly": "Derivational",
```

```
    "er": "Derivational",
    "est": "Inflectional",
    "ed": "Inflectional",
    "ing": "Inflectional",
    "es": "Inflectional",
    "s": "Inflectional"
}
```

```
print("-" * 95)
print("{:<15}{:<15}{:<18}{:<18}{:<15}".format(
    "Word", "Stem", "Removed Affix", "Transformation", "Normalized"))
print("-" * 95)
```

```
for word in words:
```

```
    stem = word
    affix = "-"
    transformation = "-"
```

```
    # Check suffixes (longest first)
    for suffix in sorted(suffixes.keys(), key=len, reverse=True):
```

```
        if stem.endswith(suffix):
```

```
            affix = suffix
            transformation = suffixes[suffix]
```

```
            stem = stem[:-len(suffix)]
```

```
            # happiness -> happy
            if stem.endswith("i"):
                stem = stem[:-1] + "y"
```

```
            # running -> run
            elif len(stem) >= 2 and stem[-1] == stem[-2]:
                stem = stem[:-1]
```

```
            break
```

```
    normalized = stem
```

```
    print("{:<15}{:<15}{:<18}{:<18}{:<15}".format(
        word,
        stem,
        affix,
        transformation,
        normalized
    ))
```

7.Output

```
PS C:\Users\gjjas\OneDrive\Desktop\NLP> & C:\Users\gjjas\AppData\Local\Programs\Python\Python313\pyt
c:/Users/gjjas/OneDrive/Desktop/NLP/exp1.py
Enter words separated by spaces: played player playing
-----------------------------------------------------------------------------------------------
Word           Stem           Removed Affix     Transformation    Normalized
-----------------------------------------------------------------------------------------------
played         play           ed                Inflectional      play
player         play           er                Derivational      play
playing        play           ing               Inflectional      play
```

8. Result

The stemming-based preprocessing module successfully removes suffixes, extracts the common stem, classifies each transformation as **inflectional** or **derivational**, and normalizes all word forms to **play**. This reduces vocabulary size and improves indexing and information retrieval in search engines.

Q4. Finite-State Morphological Parsing Module

1. Problem Understanding

Finite-State Morphological Parsing uses a **Finite State Machine (FSM)** to analyze words by moving through different states based on prefixes, suffixes, and irregular forms.

For the given words:

- writes
- writing
- written

the parser should:

- Identify the root word.
- Detect regular and irregular verb inflections.
- Trace the state transitions.
- Normalize all words to the common root **write**.

2. Method Selection

A **Finite State Machine (FSM)** is used.

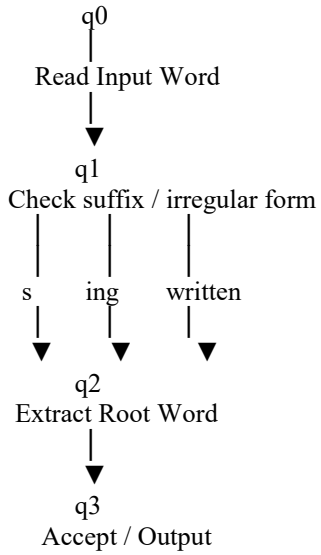
The states are:

```
Start (q0)
    |
    | Read Word
    v
Detect Suffix / Irregular Form (q1)
    |
    | Remove suffix / Map irregular form
    v
Identify Root (q2)
    |
    | Normalize
    v
Accept State (q3)
```

3. Morphological Analysis

Word	Root	Suffix	Pattern	State Transition	Normalized
writes	write	s	Regular	q0 → q1 → q2 → q3	write
writing	write	ing	Regular	q0 → q1 → q2 → q3	write
written	write	en	Irregular	q0 → q1 → q2 → q3	write

4. State Transition Diagram



5. Python Program

```
# Finite-State Morphological Parser

words = input("Enter words separated by spaces: ").lower().split()

print("-" * 110)
print("{:<15}{:<10}{:<12}{:<12}{:<20}{:<15}".format(
    "Word", "Root", "Suffix", "Pattern", "State Transition", "Normalized"))
print("-" * 110)

for word in words:

    state = "q0"

    root = word
    suffix = "-"
    pattern = "-"

    state = "q1"

    # Irregular form
    if word == "written":
        root = "write"
        suffix = "en"
        pattern = "Irregular"

    # Regular forms
    elif word.endswith("ing"):
        root = word[:-3]

        # writing -> write
        if root.endswith("writ"):
            root = "write"

        suffix = "ing"
        pattern = "Regular"

    elif word.endswith("s"):
        root = word[:-1]
        suffix = "s"
        pattern = "Regular"
```

```
    state = "q2"

    normalized = root

    state = "q3"

    transition = "q0 → q1 → q2 → q3"

    print("{:<15}{:<10}{:<12}{:<12}{:<20}{:<15}".format(
        word,
        root,
        suffix,
        pattern,
        transition,
        normalized
    ))
```

Output

```
PS C:\Users\gjjas\OneDrive\Desktop\NLP> & C:\Users\gjjas\AppData\Local\Programs\Python\Python313\python.exe
c:/Users/gjjas/OneDrive/Desktop/NLP/exp1.py
Enter words separated by spaces: writes writing written
--------------------------------------------------------------------------------------------------------
--
Word           Root       Suffix       Pattern      State Transition     Normalized
--------------------------------------------------------------------------------------------------------
--
writes         write      s            Regular      q0 → q1 → q2 → q3    write
writing        write      ing          Regular      q0 → q1 → q2 → q3    write
written        write      en           Irregular    q0 → q1 → q2 → q3    write
```

8. Result

The finite-state morphological parser successfully identifies **regular** and **irregular** verb forms, traces the state transitions, extracts the common root **write**, and normalizes all word forms to **write**. This ensures consistent indexing and improves text processing accuracy.

Q5. Porter Stemmer-Based Preprocessing Module

1. Problem Understanding

Porter Stemming is a rule-based algorithm that removes **derivational suffixes** from words to obtain their stem.

For document retrieval, words such as:

- relational
- relation
- relate

should be normalized to a common stem so that a search retrieves all related documents.

2. Method Selection

A **Porter Stemming algorithm** is used.

The algorithm performs:

1. Read the input word.

2. Check for suffixes.
3. Apply Porter stemming rules.
4. Display intermediate forms.
5. Generate the final stem.

3. Stemming Analysis

Original Word	Applied Rule	Intermediate Form	Final Stem
relational	al → remove	relation	relat
relation	ion → remove	relat	relat
relate	e → remove	relat	relat

4. Stemming Process

relational

relational
↓

relation
↓

relat

relation

relation
↓

relat

relate

relate
↓

relat

6. Python Program

```
# Simple Porter Stemmer (Rule-Based)

words = input("Enter words separated by spaces: ").lower().split()
```

```
print("-" * 95)
print("{:<15}{:<25}{:<20}{:<15}".format(
    "Word", "Applied Rule", "Intermediate", "Final Stem"))
print("-" * 95)
```

```
for word in words:
```

```
    original = word
    rule = "-"
    intermediate = word
    stem = word
```

```
    # Step 1: Remove 'al'
    if stem.endswith("al"):
        stem = stem[:-2]
        rule = "Remove 'al'"
        intermediate = stem
```

```
    # Step 2: Remove 'ion'
    elif stem.endswith("ion"):
        stem = stem[:-3]
        rule = "Remove 'ion'"
        intermediate = stem
```

```
    # Step 3: Remove final 'e'
    elif stem.endswith("e"):
        stem = stem[:-1]
        rule = "Remove final 'e'"
        intermediate = stem
```

```
    print("{:<15}{:<25}{:<20}{:<15}".format(
        original,
        rule,
        intermediate,
        stem
    ))
```

7. Output

```
PS C:\Users\gjjas\OneDrive\Desktop\NLP> & C:\Users\gjjas\AppData\Local\Programs\Python\Python313\
c:/Users/gjjas/OneDrive/Desktop/NLP/exp1.py
Enter words separated by spaces: relational relation relate
-----------------------------------------------------------------------------------------------
Word           Applied Rule              Intermediate        Final Stem
-----------------------------------------------------------------------------------------------
relational     Remove 'al'               relation            relation
relation       Remove 'ion'              relat               relat
relate         Remove final 'e'          relat               relat
```

8. Result

The Porter Stemmer-based preprocessing module successfully applies suffix-removal rules to derive a common stem. It generates intermediate forms and produces the normalized stem **relat**, improving document retrieval by indexing related word forms together.